

PIEDPIN

Let's try CISSP!

なぜCISSPを学ぶのか

- ✓ CISSP(Certified Information Systems Security Professional)とは、世界標準の**セキュリティ・プロフェッショナル認定**です。
- ✓ (ISC)²（国際情報システムセキュリティ認証コンソーシアム、International Information Systems Security Certification Consortium）が認定を行っています。
- ✓ 情報セキュリティ専門家が共通言語として利用している『(ISC)² CISSP CBK』を理解している者に与えられる資格です。
- ✓ 2004年6月に、米国規格協会（ANSI）より**ISO/IEC17024**の認証を受け、信頼度がより高くなりました。（https://japan.isc2.org/cissp_about.html）
- ✓ 2022年7月1日現在、日本のCISSPは**3,699**人です。

(ISC)² が展開する資格ソリューション



国際的に認められた情報セキュリティ・プロフェッショナル認定資格です。米国規格協会（ANSI）よりISO/IEC17024の認証を受けた。



ベンダーフリー・カントリーフリーの情報セキュリティの資格です。情報セキュリティに従事しないが、組織を理解し、適切なコミュニケーションを行う。



クラウドサービスを安全に利用するために必要な知識を体系化した資格です。



ソフトウェアの脆弱性に端を発したセキュリティインシデントの増加や、政府や重要インフラにおけるセキュアなソフトウェアニーズに対応する資格です。

(ISC)² CISSP CBK ドメイン



ドメイン 1
セキュリティとリスク
マネジメント



ドメイン 2
資産のセキュリティ



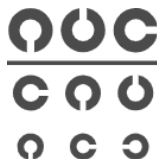
ドメイン 3
セキュリティアーキ
テクチャとエンジニ
アリング



ドメイン 4
通信とネットワー
クのセキュリティ



ドメイン 5
アイデンティティと
アクセスの管理



ドメイン 6
セキュリティの評
価とテスト



ドメイン 7
セキュリティの運用



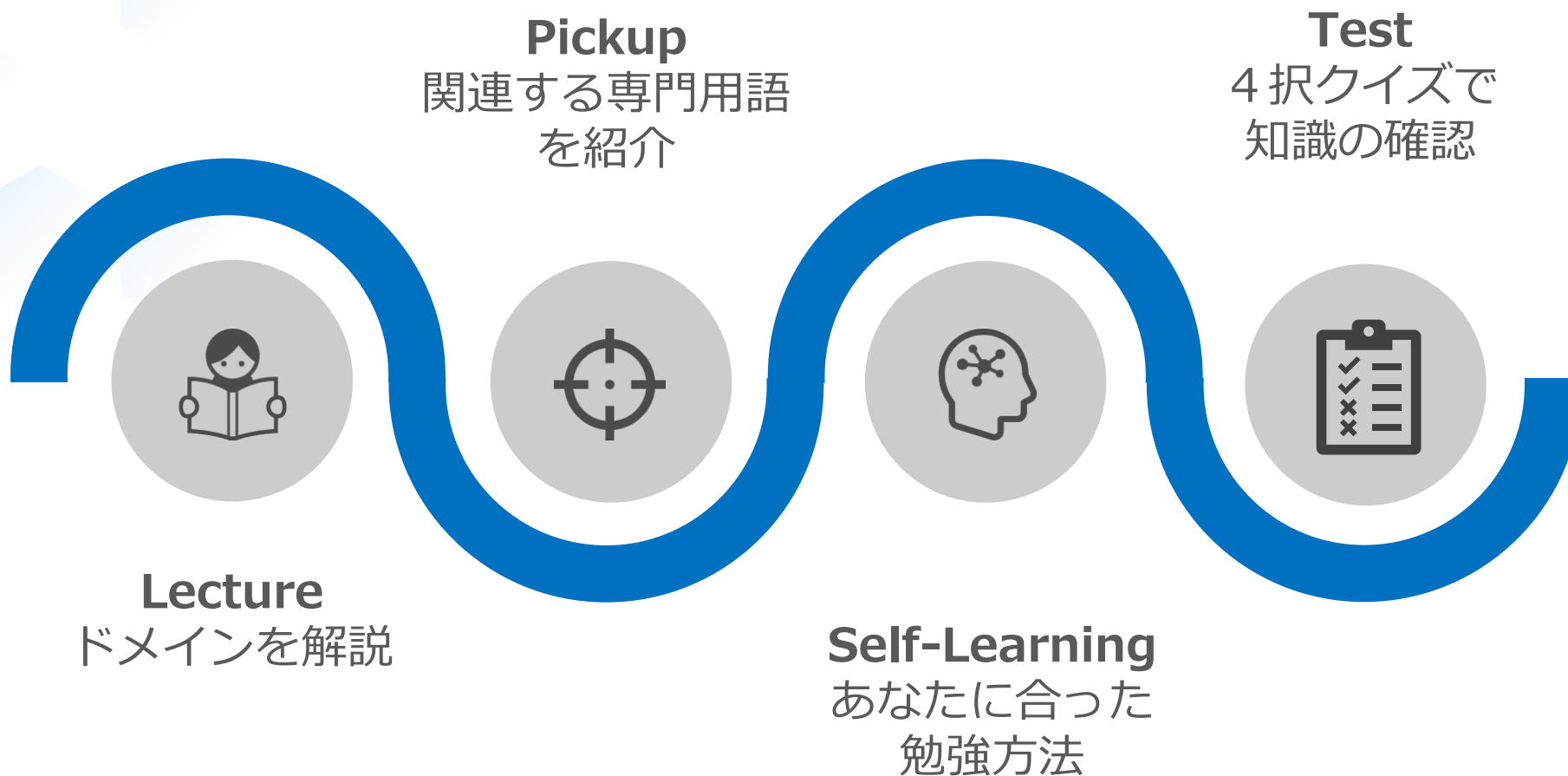
ドメイン 8
ソフトウェア開発
セキュリティ

あなたのゴールと本レッスンの内容

- ✓ 情報セキュリティ初心者のために設計されたCISSP講座
経験者からすると“当たり前”と感じる内容も含まれます。
- ✓ 本レッスンの対象は、CBKのドメイン 6 です。
他のドメインのレッスンは対象外となります。
- ✓ CISSP合格のためのアップグレードが必要です。
現代のセキュリティの関心、流行、考え方を織り込みます。



レッスンの流れ



ドメイン別試験比重



ドメイン 1
セキュリティと
リスクマネジメント

16%



ドメイン 2
資産のセキュリティ

10%



ドメイン 3
セキュリティアーキテク
チャとエンジニアリング

13%



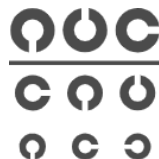
ドメイン 4
通信とネットワーク
のセキュリティ

13%



ドメイン 5
アイデンティティと
アクセスの管理

13%



ドメイン 6
セキュリティの評価
とテスト

12%



ドメイン 7
セキュリティの運用

13%



ドメイン 8
ソフトウェア開発
セキュリティ

10%



監査

監査

セキュリティ評価

● セキュリティ機能を確認する目的

- 組織は、自身の運営や情報技術が想定通りに機能しているかを確認する必要があります。
- ポリシーや手順といった管理的な統制やITシステムを含め、セキュリティ評価の対象範囲は多岐にわたります。
- 定期的に組織と情報技術の管理体制の適切性や有効性を評価し、セキュリティリスクを事前に察知して対応します。

項目	セキュリティ評価	セキュリティ監査	セキュリティテスト
対象	 ITシステムとそれを管理する 組織全体の体制	 プロセス、手続き、規制への 準拠	 ソフトウェア、ネットワーク、シ ステム
目的	セキュリティ体制の有 効性	ポリシーや規制への 準拠	システムやアプリケー ションの脆弱性検出
主な基準/ガ イドライン	NIST、ISO、内部 評価基準	ISO 27001、SOX、 内部統制基準	OWASP、CWE、 SANS 25 など

監査

セキュリティ評価

●セキュリティ監査

項目	内部監査	外部監査	サードパーティ監査
目的	内部統制の強化	規制や会計基準への適合性	第三者認証
実施者	組織内の監査部門 または従業員	独立した外部の監査 機関または公認会計 士	認証機関や専門の 第三者機関
主な基準/フレームワーク	内部ポリシー、ISO内 部監査手順など	会計基準（例： IFRS、GAAP）、法 令（例：SOX法）	国際標準（例： ISO 27001、SOC 2）

プロセス改善の内部監査、社会的義務の外部監査、自慢できるサードパーティ監査と考えましょう。

評価対象との独立性

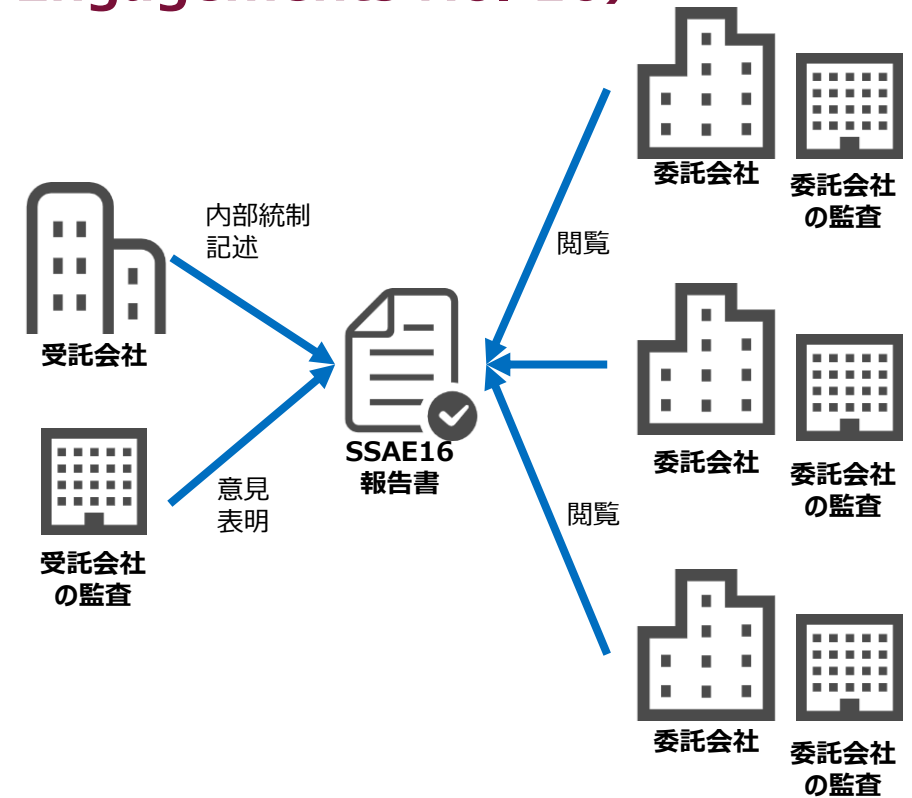
CISSPの試験では、評価する側と評価される対象の独立性の観点が目されます。これは、セキュリティ監査だけではなく、情報システムへのテストを実施する際にも体制の独立性が求められます。

BIG4監査法人の企業

BIG4とは、世界的に会計事務所と連携している監査法人が立ち上げた税理士法人になります。Deloitte（デロイト）、PwC（PricewaterhouseCoopers, PwC）、EY（Ernst & Young）、KPMGがBIG4です。大手企業や外資系企業から中小企業までさまざまなクライアントに税務会計だけではなく、監査業務を行っています。

●SSAE16 (Statement on Standards for Attestation Engagements No. 16)

- ある監査対象組織の業務の一部を他の組織もしくは会社に委託している場合、委託先（受託会社、アウトソーシング企業）に対しても適性を確認する必要があります。
- SSAE16では、監査組織の業務の一部もしくはすべてを委託している会社に対する監査を整備しています。受託会社から内部統制に関わる評価報告書を提出してもらい、その有効性を確認します。
- SSAE16によって監査が完了すると2つのタイプのアウトプットが提出されます。Type1（時点評価）では、ある時点において、内部統制の仕組みがあることが保証されます。Type2（期間評価）では、Type1の評価に加え、一定期間に内部統制が実施されているかを保証します。



受託会社SSAE 16報告書を提出し、委託会社はその報告書を閲覧利用

監査

コンプライアンスチェック

●コンプライアンスチェック

- コンプライアンス（Compliance）とは、法律や規則、社会的規範、企業内のルールなどに適切に従うことを指します。
- コンプライアンスは単なる「ルールの順守」にとどまらず、組織や個人が社会的責任を果たし、信頼を築くための基盤です。
- セキュリティ監査は、セキュリティ体制が機能しているかを確認するのに対して、コンプライアンスチェックではルールに従っているかを確認します。

コンプライアンスチェック

規制や基準に対する指摘や改善提案



例: HIPAA、GDPR

セキュリティ監査

セキュリティの脆弱性、改善点、及びリスク評価



例: ISO 27001、NIST SP 800-53

監査

監査ログ

●セキュリティ監査ログ

取得対象ログ例

- [NIST SP 800-92](#)（コンピュータセキュリティログ管理ガイド）は、収集して監査する必要があることがログの種類が記載されています。

セキュリティ監査ログの保管方法のポイント

- ログの収集は自動的に保管されること。
- セカンダリコピーと比較することにより、データの完全性を検証すること。
- 読み取り権限のみを付与するアクセス制限されていること。
- ログの検索性を向上させること。

ネットワークセキュリティソフトウェア/ハードウェア系

ウイルス対策ログ

IDS / IPSログ

リモートアクセスソフトウェア

Webプロキシ

脆弱性管理

認証サーバー

ルーター

ファイアウォール

オペレーティングシステム系

システムイベント

監査レコード

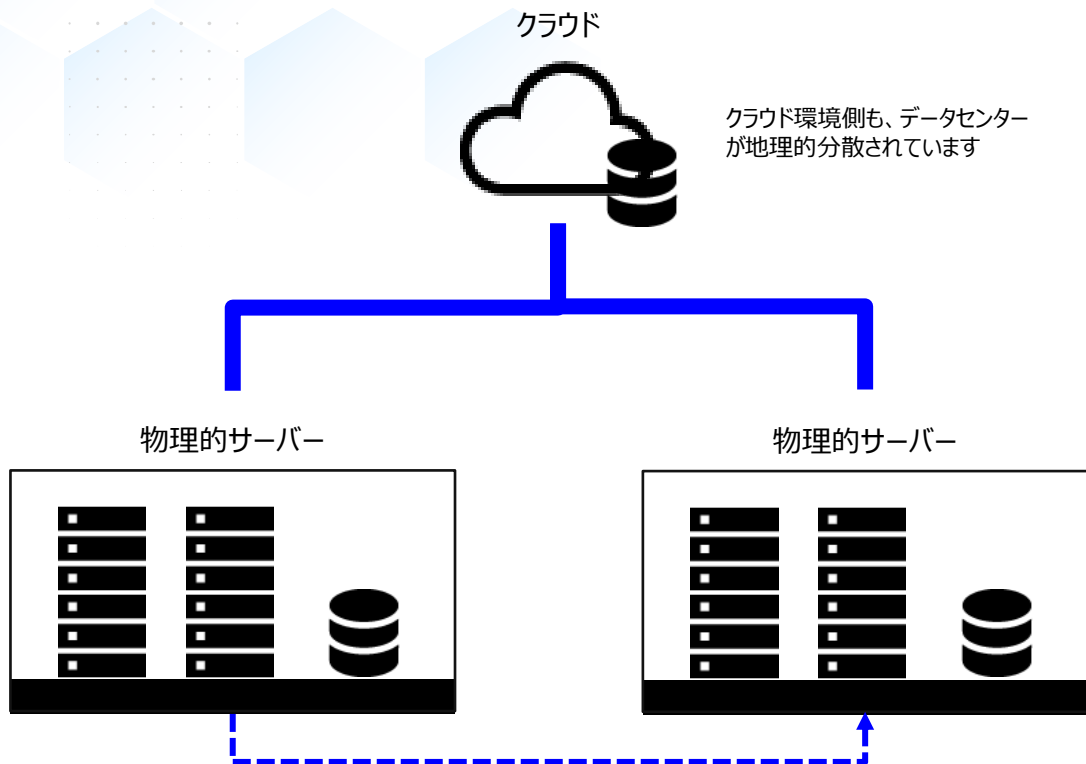
アプリケーション

クライアント要求とサーバー応答

使用状況情報

重要な運用アクション

● オンプレミス環境／クラウド環境／ハイブリッド環境での保管方法



項目	オンプレミス環境	クラウド環境	ハイブリッド環境
ログの保管場所	組織内のサーバーやストレージに保存	クラウドプロバイダーの提供するストレージサービスに保存	オンプレミスとクラウドの双方で保存 (クラウドで一括管理することも考えられます)
拡張性	ストレージ容量の増設が必要	自動的にスケールアップ可能	両環境でスケーラビリティを統一する工夫が必要
保管方法の例	ローカルディスク、NAS、SANなど	AWS S3、Azure Blob Storageなど	両環境での保管方法に則ります
ログの収集手段の例	syslog、SIEM	AWS CloudTrail、Azure Monitor	Splunk、Elastic Stack



セキュリティ対策テスト

セキュリティ対策テスト

テストの準備

● 脆弱性診断の留意事項

脆弱性診断を行う体制

- 結果の信頼性の観点から、開発を行った組織では脆弱性診断を行うことができません。第3者の立ち位置のチームが脆弱性診断を行います。

脆弱性診断の期間

- 基本的に、脆弱性診断中はシステムの更新や作業を行うことができません。
- 脆弱性を修正すると新しい脆弱性が生まれ、診断の範囲を不明瞭になる可能性があります。

脆弱性診断の規模感

- 一般的なWebアプリケーションの場合、多くの脆弱性診断を行う組織では、審査の対象となるサイトやURLの数で期間や金額を算出します。

脆弱性診断の承認

- アプリケーションの機能がクラウドサービスに依存している場合には、脆弱性診断を行うという操作自体がそのクラウド基盤への攻撃と見なされる可能性があります。
- クラウドサービス事業者が脆弱性診断や侵入テストに対して、どのような扱いをしているのかを事前に調べておく必要があります。

脆弱性に対する改修

- 脆弱性を発見し、解消したことを確認しクローズとなります。
- 脆弱性診断は機能テスト後に行われますが、開発体制を保っておく必要があります。

セキュリティ対策テスト

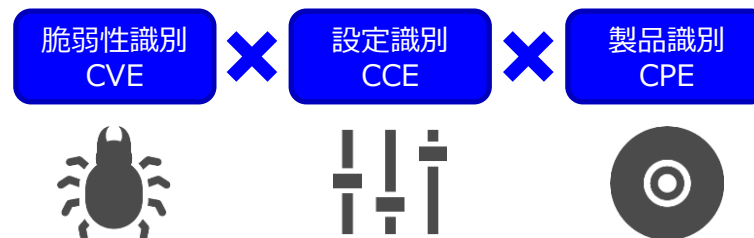
脆弱性管理

● SCAP (Security Content Automation Protocol)

- NIST (National Institute of Standards and Technology) において、情報セキュリティ対策の自動化と標準化を目指したSCAP (Security Content Automation Protocol) の開発が行われました。
- 製品で脆弱性が発生した際には、脆弱性は[NVD](#)に公開され、脆弱性対策するように促されます。

SCAPの構成要素

- CVE (Common Vulnerabilities and Exposures) とは、脆弱性に対する識別子です。
- CCE (Common Configuration Enumeration) とは、パスワード有効期限やパスワード長さなどシステム設定に対するセキュリティ設定の識別子です。
- CPE (Common Platform Enumeration) とは、製品に対する識別子です。



SCAPの構成要素

セキュリティ対策テスト

脆弱性管理

● SCAP (Security Content Automation Protocol)

- CVSS (Common Vulnerability Scoring System) とは、脆弱性の深刻度を評価するため項目です。
- XCCDF (eXtensible Configuration Checklist Description Format) とは、セキュリティチェックリストやベンチマーク（数量化されたテスト）などを記述するための仕様言語です。
- OVAL (Open Vulnerability and Assessment Language) とは、コンピュータのセキュリティ設定状況を検査するための仕様です。製品ベンダ、セキュリティベンダなどが提供する文書による脆弱性対策情報を記載します。

XCCDFとOVALの違い

あなたが管理者なら、自身のシステムに脆弱性があるかをいち早く確認したいでしょう。そのためには、テスト項目とどのようなテスト手順が必要になります。SCAPでは、文章ではなくシステムが読み込めるXML形式で記載します。こうすることで、脆弱性診断ツールに読み込ませいち、ツールが人の手で探すよりも早く脆弱性を見つけることが可能になるわけです。

XCCDFとOVALは、いずれもXML形式の記載の仕方です。XMLはタグを自由に定義できますから、「このタグの時にはこの意味」という紐づけが必要なのです。

各テスト項目が記載するところにXCCDFで記述し、実際にどのような検証を行うかはOVALで記述します。

セキュリティ対策テスト

脆弱性分析

●脆弱性診断

- 脆弱性は製品の持つ脆弱性だけではなく、個別のシステム構成や設定によって発生する脆弱性もあります。
- 脆弱性診断/脆弱性検査ツールを使用して、ネットワークまたはシステムをスキャンし、システムの設定ミスや古いソフトウェア、パッチ欠如など、事前定義された脆弱性のリストを探します。

最新の脆弱性を把握しておかないと合格できない？

試験対策において、最新の脆弱性内容を把握する必要はありません。CISSPの試験問題は定期的に変更され、最新のトレンドを織り込みますが、詳細でテクニカルな内容は聞かれません。ある特定のふるまいをする脆弱性に対してどのような扱いをしなければならないのかという観点から出題されます。

セキュリティ対策テスト

脆弱性分析

● 脆弱性診断ツール

Nessus

- Nessusは、コンピュータセキュリティに対する包括的な脆弱性診断ツールであり、世界で最も使われています。
- Tenable Network Security社により、開発支援されています。

OpenVAS

- OpenVAS (Open Vulnerability Assessment System) は、脆弱性診断ツールNessusから派生した製品です。
- Greenbone Networks社により、開発支援されています。
- 未認証および認証が必要なテスト、高レベルおよび低レベルのインターネットおよび産業用通信プロトコルに対応、大規模スキャンをするためのパフォーマンスチューニング機能、内部プログラミング言語などを含みます。

セキュリティ対策テスト

脆弱性分析

● 脆弱性診断の対象

ネットワーク



nmapなどのツールを用いてポートスキャンを実施し、不要なポートが開放されていないか、またはポートが適切に閉鎖されているかを確認します。

アプリケーション



変更が頻繁に行われる公開アプリケーションは、特に慎重な管理が求められます。新規導入時には包括的な脆弱性診断を実施し、変更後も再度診断を行い安全性を確認しましょう。

データベース



システム内で重要な機構であるため、外部からの直接アクセスを防ぐことはもちろん、外部向けWebアプリケーションを介したSQLインジェクションなどの攻撃対策にも十分注意を払う必要があります。

セキュリティ対策テスト

脆弱性分析

● 共通脆弱性評価システム (CVSS、Common Vulnerability Scoring System)

基本評価基準：脆弱性そのものの特性を評価する基準

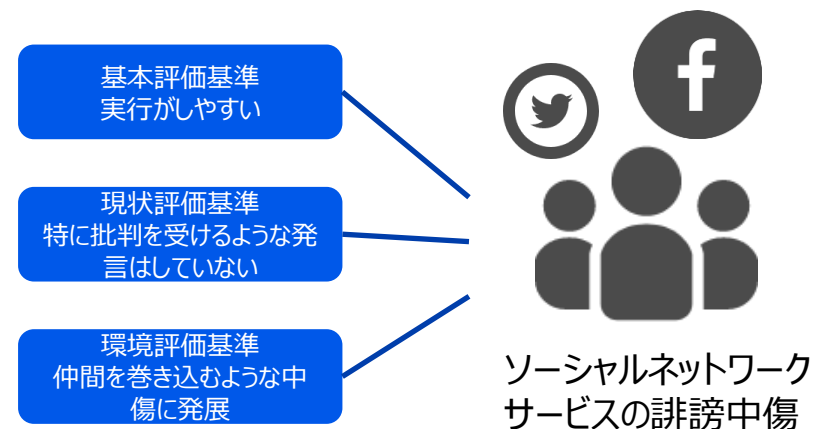
- AV(Access Vector)：攻撃元区分
- AC(Access Complexity)：攻撃条件の複雑さ
- Au(Authentication)：攻撃前の認証要否
- C(Confidentiality Impact)：機密性への影響
- I(Integrity Impact)：完全性への影響
- A(Availability Impact)：可用性への影響

現状評価基準：脆弱性の現在の深刻度を評価する基準

- E (Exploitability)：攻撃される可能性
- RL(Remediation Level)：利用可能な対策のレベル
- RC (Report Confidence)：脆弱性情報の信頼性

環境評価基準：利用者を含め、最終的な脆弱性の深刻度を評価する基準

- CDP(Collateral Damage Potential)：二次的被害の可能性
- TD(Target Distribution)：影響を受ける対象システムの範囲
- CR, IR, AR (Security Requirements)：対象システムのセキュリティ要求度



セキュリティ対策テスト

ペネトレーションテスト

● ペネトレーションテスト（ペンテスト、倫理的ハッキング）

- システムの脆弱性が悪用可能かどうかをテストします。
- セキュリティの弱点を探し、システム、建物、およびデータへのアクセスを取得する可能性のある、組織やシステムに対する許可されたシミュレートされた攻撃です。

脆弱性診断

情報システムに存在するセキュリティ問題である脆弱性を見つけることが目的

広く、浅い

OpenVAS

ペネトレーションテスト

攻撃者としてのゴールを設定し、それを実現できるかどうかを検証することが目的

狭く、深い

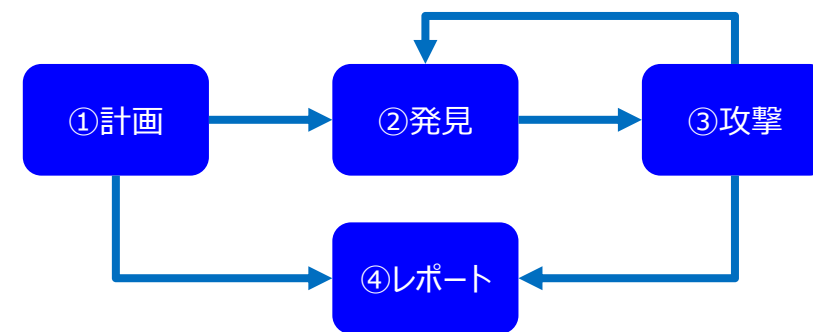
Metasploit Framework

セキュリティ対策テスト

ペネトレーションテスト

● ペネトレーションテストの手順

- ① 計画：IP範囲、時間枠、ツール、POC、テスト方法、テスト対象を作業範囲記述書で定義し承認をもらいます。また、ペンテスター（ペネトレーションテストを行う人）を採用する前に、作業範囲記述書内容を法務チームに確認します。
- ② 情報収集と発見：ペンテスターは攻撃対象の情報を収集し、脆弱性を発見します。
- ③ 攻撃：実際にネットワークにアクセスし、侵入を試みます。より高いレベルの権限を取得し、最終的には管理者アクセスも試みます。内部に攻撃ツールをインストールするなどして新しい攻撃対象領域に展開します。
- ④ レポート：テストが完了したら、調査結果を報告します。ペンテスターは、脆弱性を修正するのではなく、レポートとして文書化します。



ペネトレーションテストは攻撃から追加の脆弱性を発見するケースもある

セキュリティ対策テスト

ペネトレーションテスト

● ペネトレーションツール

ペネトレーションの発見ツール

- オープンソースインテリジェンス（OSINT、Open Source Intelligence）とは、一般に公開され利用可能な情報を源に、機密情報を収集する手法です。企業の公式サイトや公開している営業資料などを収集します。
- [theHarvester](#)は、パッシブな調査をメインにして、電子メール、従業員名、バナーなど情報を検索エンジン、PGPキーサーバーから情報収集を行います。

ペネトレーションの攻撃ツール

- システムやアプリケーション固有の脆弱性を見つけるよりも実際の攻撃を再現することを目的としています。
- [Metasploit Framework](#)は、攻撃者の行動をシミュレートしペネトレーションテストを行うソフトウェアです。世界で最も普及しているペネトレーションフレームワークです。

セキュリティ対策テスト

ペネトレーションテスト

● ペンテスター



ホワイトボックス・ペンテスト (フル・ナレッジ)

- ペンテスターは、内部ネットワークに関する知識を持ちます。
- 管理者アクセスできる従業員のように当社の環境について完全な知識を持っていることを想定します。
- 脆弱性を早く見つけられますが、それは十分な知識によって到達し得る発想であるとも言えます。

グレーボックス・ペンテスト (パーティカル・ナレッジ)

- ペンテスターは、限られた知識しか持っていません。
- 関連ベンダーなどの限られた環境の知識を持つ人を想定しています。
- 現実的ではないホワイトボックステストと時間のかかり過ぎるブラックボックステストのデメリットを補うための中間的なテストです。

ブラックボックス・ペンテスト (ゼロ・ナレッジ)

- ペンテスターは、公開されている情報以外には組織について何の知識も持っていません。
- 外部の悪意のある攻撃者をシミュレートすることができます。
- しかし、その分脆弱性を発見することは至難であり、結果が得られるまでホワイトボックス・ペンテストよりも時間を要します。

セキュリティ対策テスト

違反攻撃シミュレーション（BAS）

● 侵害攻撃シミュレーション（BAS、Breach and Attack Simulation）

- 複雑なサイバー攻撃をシミュレートする自動化ツールです。
- ペネトレーションテストでは基本的に専門家の手動による検査方法も含まれますが、BASでは自動化されたツールであることが強調されます。
- 目的はペネトレーションテストと同じですが、自動化による恩恵を受けます。攻撃手法の網羅性やツールとしての稼働時間、それに加えて完全にシミュレートされた攻撃手法が特徴的です。

オープンソースに対するテストは必要？

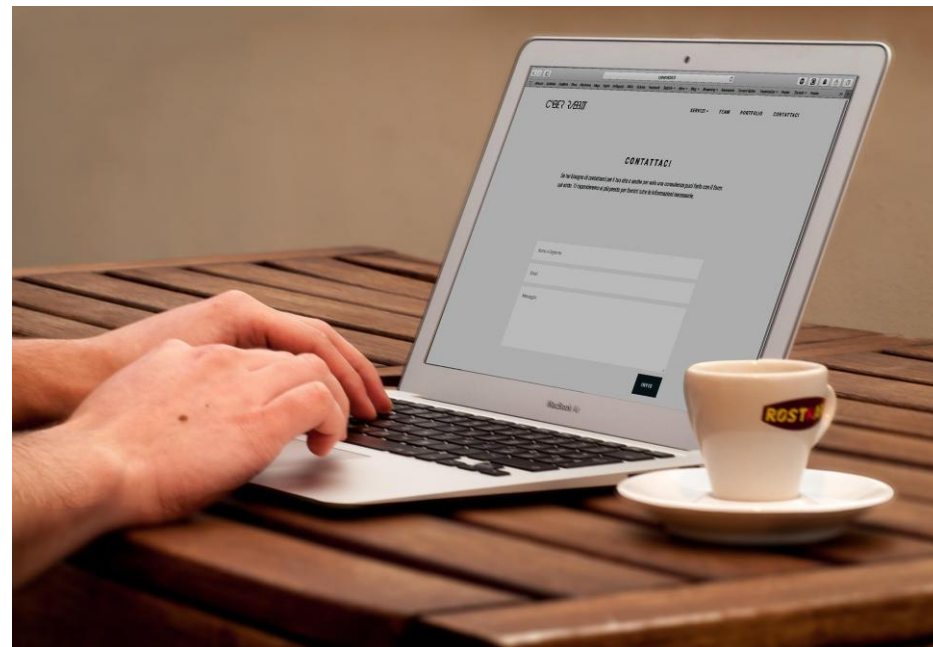
OSSTMM（Open Source Security Testing Methodology Manual）とは、オープンソースのペネトレーションテストの標準です。オープンソースは、無料で誰でも使えることからエクスプロイト攻撃などの対象になりやすいことから、信頼が低い見方があります。しかし、ちゃんとリスクを理解していればこれほど素晴らしいものはありません。オープンソースに対して標準的なテストを行い、実装することを検討してみてもいいでしょうか？

セキュリティ対策テスト

悪用ケーステスト

●悪用ケーステスト（Misuse Case Testing）

- システムに対して悪意のある行為を実行しても、攻撃者は通常のユーザーと同じようには行動しません。
- 悪用ケーステストは、セキュリティパッチの穴を付くような脆弱性検査ではなく、正常な動作を前提とし変な使い方しても予期せぬ動きをしないかを確認します。
- 例 1：アプリケーションではフォームを送信する前に特定のフィールドを必須で要求することがありますが、このフィールドが空白のままの送信された場合、空白のフィールドに記入するよう要求するかどうかを確認します。
- 例 2：郵便番号を入力するフォームに対して文字列や名前を入力するフォームに対して記号を埋め込んで送付した場合の動作を確認する。
- 例 3：ログイン後の画面のURLを記憶して置き、ログインをせずに直接そのURLを打鍵した場合の動作を確認する。



ユーザーは意図的にではなく間違えるケースもあるためUIの精度を確かめるテストでもある



ソフトウェア動作テスト

ソフトウェア動作テスト

テストの目的

●ソフトウェアテストの重要性

アクセス権の超過



ソフトウェアアプリケーションは、オペレーティングシステム、ハードウェア、その他リソースに対して特権のアクセス権を持ちます。

情報資産の漏洩



ソフトウェアアプリケーションは、クレジットカード番号、社会保険番号やビジネス資産などの重要な情報を取り扱います。

ビジネス停止



ソフトウェアアプリケーションの欠陥は、ビジネスシステムに処理により非常に大きな影響を与えます。

ソフトウェア動作テスト

テストの計画

●ベンチマーク

- ソフトウェアテストにおけるベンチマークとは、特定の基準や指標を設定し、それに基づいてソフトウェアの性能や品質を測定・評価するプロセスや手法を指します。
- ベンチマーク結果を分析することで、ソフトウェアのどの部分が改良の余地があるかを特定します。

パフォーマンステスト



応答時間、スループット、リソース使用量などを測定し、目標値や競合の性能と比較します。

負荷テスト



特定の条件下でのシステムの耐久性や限界を評価します。

セキュリティテスト



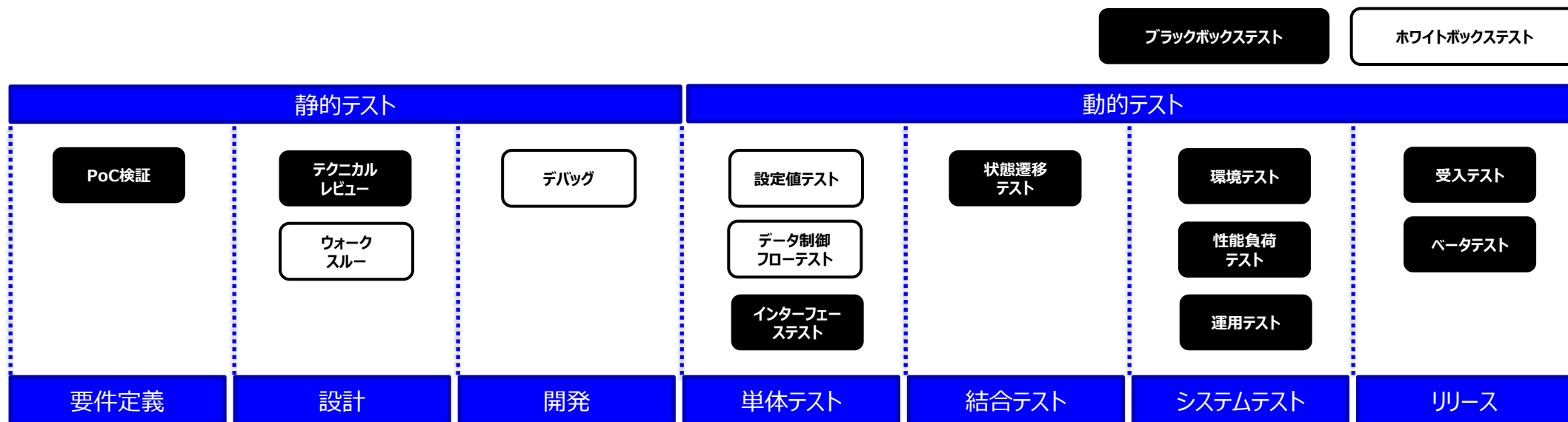
業界標準（例：OWASP Top 10）の脆弱性評価基準を用いて、セキュリティの強度を測定します。

ソフトウェア動作テスト

テストの計画

●ソフトウェアテストの区分

- 要件定義から開発工程までは、打合せや資料などを用いた認識合わせが主になります。
- 開発工程では、ブラックボックステストとなる項目はありません。コーディングした際にはブラックボックスとなる観点がないためです。
- リリース後もテストがあります。実際のユーザーに使っていただき、使用感も含めた検証を行います。
- テストの種類やテスト範囲はプロジェクトごとに異なります。



ソフトウェア動作テスト

テストの計画

● 静的テスト

- 静的テストとは、ウォークスルー、構文チェック、コードレビューなどの実行していないコードのテストです。
- ソースコード自体を見て、既知の安全でない手法、関数、ライブラリ、またはその他の特性がソースコードに使用されていない箇所を探します。
- いくつかの開発環境では、プログラミング言語に対して静的コード解析ツールが包含されており、デバッグ時にエラー箇所とともに脆弱性に当たる箇所を提示してくれます。

ペアテスト

- ペアテストでは、ソフトウェアアプリケーションの開発者とそれ以外のレビュアーによって開発を進めていきます。
- レビュアーは開発者よりも上級のプログラマーを採用されます。

静的テスト

実行せずに
解析

ペアテスト、ファークン
テストなど

ログに機密情報が入っ
ています。



動的テスト

実行して
解析

Fuzzing (Fuzzテスト)、
インターフェーステスト、悪
用ケーステスト、リアルユー
ザーモニタリングなど

想定外のレスポンスが
返ってくる。



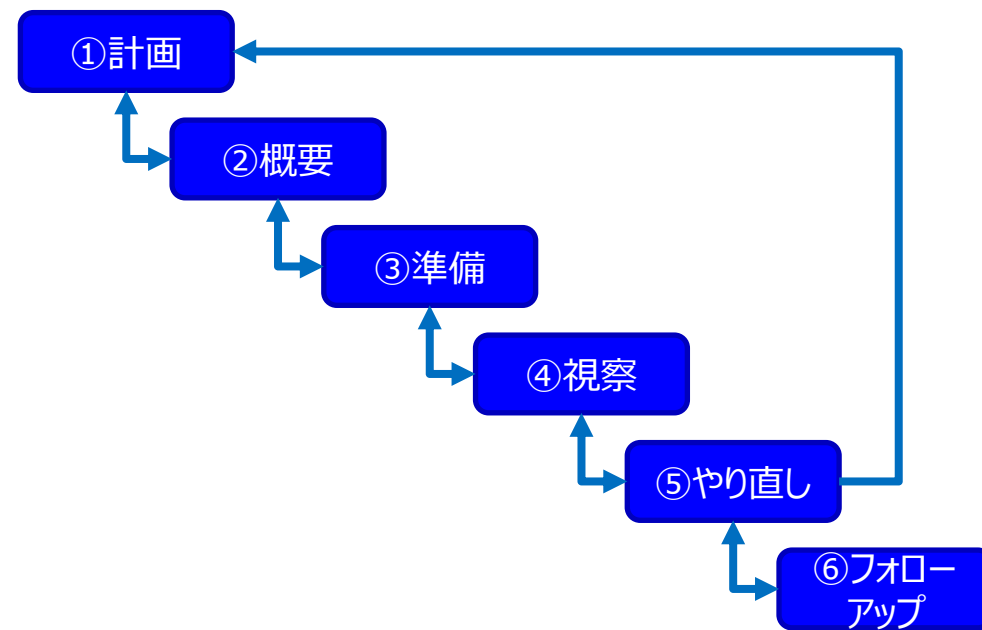
ソフトウェア動作テスト

テストの計画

● 静的テスト

Fagan検査

- Fagan検査とは、計画からフォローまで通したドキュメントレビューのプロセスです。
- 指摘を繰り返し発見することで高品質な成果物を目指すプロセスです。Fagan検査が適切に実装されていれば、明確で正確、理解しやすい、標準に準拠したドキュメントが得られるでしょう。
- Fagan検査は①計画、②概要、③準備、④視察、⑤やり直し、⑥フォローアップの6つのプロセスから成り立ちます。ステップ④の視察までは通常と同じですが、もしここでバグが見つかった場合はステップ⑤のやり直しにおいて修正した後、ステップ①からやり直しになります。



フェイガン検査はやり直しフェーズにおいて問題があった場合には計画からやり直しになる

ソフトウェア動作テスト

テストの計画

● 動的テスト

- 動的テストとは、HTTPリクエストを送付するなど実行中のコードをテストします。
- クロスサイトスクリプト、SQLインジェクションなどのウェブアプリケーション特有の脆弱性を脆弱性ツールで検出も動的テストに当たります。
- 動的テストは静的テストの上位互換ではありません。見るべき観点が異なります。そのため、静的テストも動的テストも両方実施する必要があります。

Fuzzing (Fuzzテスト)

- Fuzzテストとは、大量の異なる入力を与えて、不正なアクセスを引き起こしたり、アプリケーションが予測不可能な状態にしたりすることを試みるテストです。
- Fuzzテストは攻撃手法がシンプルですので、結果もシンプルになります。クラッシュやハングアップする結果になります。
- Mutating Fuzzingとは、実際に入力値となりえる値をFuzzテストの入力として扱うものです。そのため、ある程度想定しうる入力になります。
- Generational Fuzzingとは、論理的に入力値となりえる値をFuzzテストの入力として扱うものです。そのため、機械的かつ網羅的な入力になります。

ソフトウェア動作テスト

テストの工程

●テスト工程

単体テスト (Unit Testing)

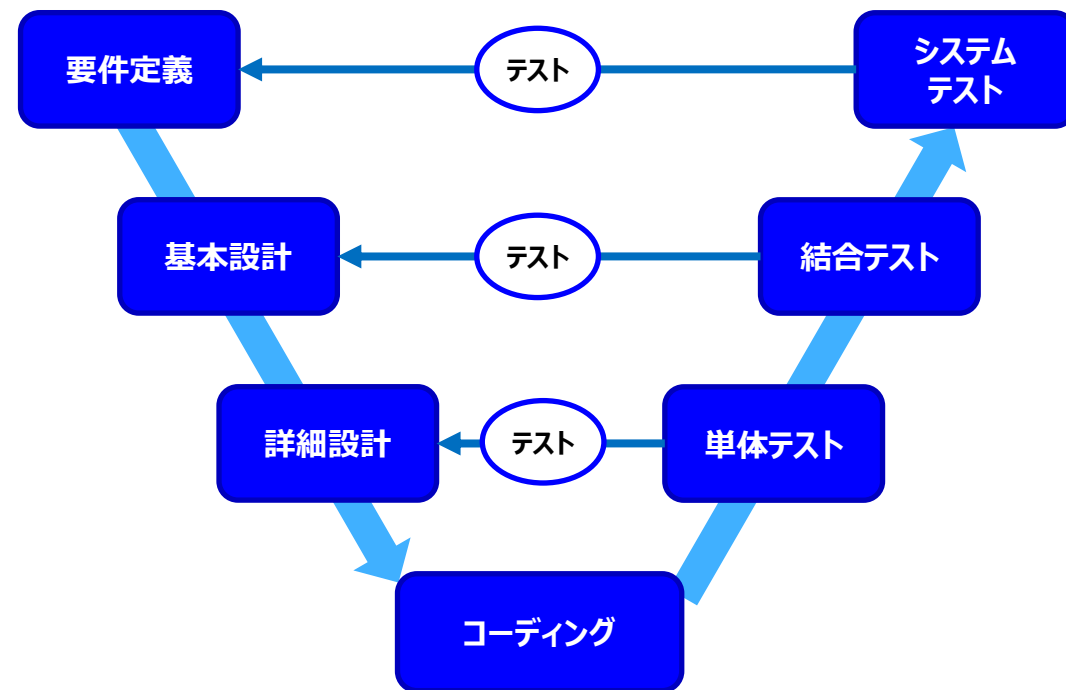
- 単体テストとは、開発したモジュールが単体で動くことを確認するテストです。モジュールとは、ソフトウェアの最小単位です。
- 詳細設計書で定義された内容に対するテストであり、各パラメータがシステムに読み込まれ、処理することを保証します。

結合テスト (Integration Testing)

- 結合テストとは、開発したモジュール同士を連結して全体動作を確認するテストです。
- 基本設計書で定義された内容に対するテストであり、定義されたシステム構成上で動作することを保証します。

システムテスト (System Testing)

- システムテストとは、提示された要件通りに開発されているかを確認するテストです。



各設計フェーズに対するテスト種類

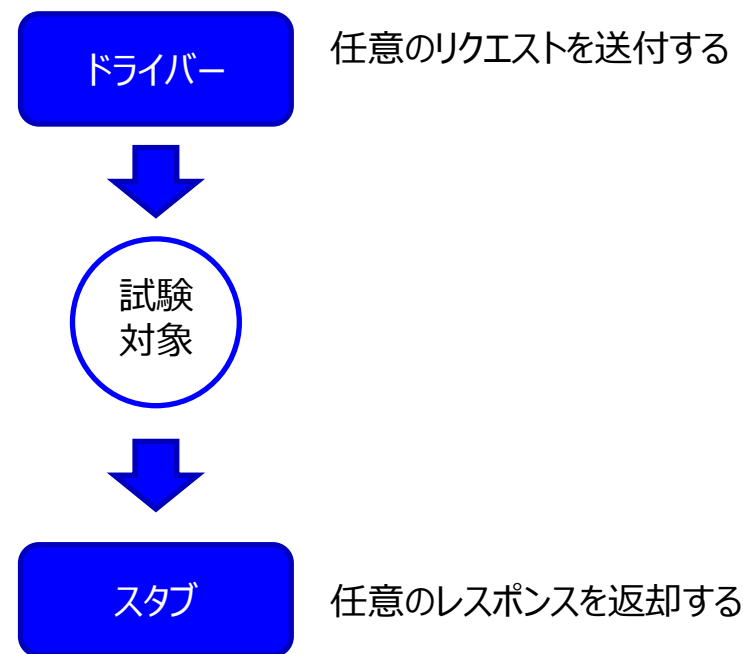
ソフトウェア動作テスト

テストの工程

●テスト工程でのテスト例

インターフェーステスト

- システムは一つのプログラムで動作することはありません。別のコンポーネントの様々なプログラムと相互に情報を交換し、処理を成立させます。
- アプリケーションのやり取り口（インターフェース）に対するテストを行います。
- インターフェーステストをする際には、ドライバーとスタブが作ることもあります。結合する予定のモジュールが開発環境にない場合には、リクエストを送付する仮のモジュールと、レスポンスを返却する仮のモジュールが必要になります。ドライバーはリクエストを送付するもので、スタブは試験対象のモジュールからのリクエストにレスポンスを返すものです。



ソフトウェア動作テスト

テストの工程

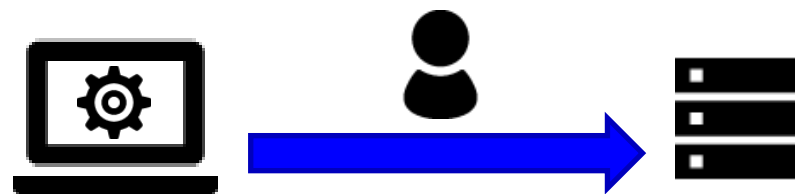
●リリース工程でのテスト例

導入テスト (Installation Testing)

- 導入テストとは、開発したモジュールがちゃんとインストールできるかを確認するテストです。主に、導入するための手順書の確立がゴールになります。
- 昨今では、クラウドの安定したアプリケーションデプロイ DevOpsソリューションにより、導入テストがリスクが少なく、簡単になっています。

回帰テスト (Regression Testing)

- 回帰テストとは、修正したことではほかの箇所に不具合が出ていないかを確認するテストです。
- プログラムの改修は、積み木を重ねる作業ではありません。新しい機能に即した全体の構成を見直す場合もあります。もしくは、変更したインターフェースが他のモジュールに即していない場合もあります。



導入テストでは、開発したモジュールを実運用のサーバーにデプロイし、正常な動作ができることまでを確認する。



回帰テストでは、改修したはずが、他の機能が使えなくなっていることもある。

ソフトウェア動作テスト

テストの工程

●リリース工程でのテスト例

リアルユーザーモニタリング (Real User Monitoring)

- リアルユーザーモニタリングとは、一般ユーザーが正しい操作をしていることを確認することです。リリース後に行うテストの一つです。
- 精度の高いシステムテストしてもサービスを開始するまではすべて仮定であり、実際何が起こるかわかりません。
- 一般的に、一部機能を制限しながら実際にサービスを公開し、リアルユーザーつまり一般ユーザーに実際に使ってもらい、問題の影響を抑えた上でテストします。
- 実際のところ、ソフトウェアの動作を確認するというよりは、マーケティング活動の一環としてユーザーの動向をリサーチする目的で使われます。

ソフトウェア動作テスト

テストの設計

●要件網羅

TM/RTM (Requirements Traceability Matrix)

- トレーサビリティマトリクスとは、顧客の要求をテスト計画にマッピングするために使用される表です。
- 現在のプロジェクトの要求が満たされているかどうかを確認したり、提案依頼書、ソフトウェア要求仕様書、各種成果物のドキュメント、プロジェクト計画のタスクなどの作成にも役立ちます。

Req. ID	Req. Desc	Senario	TC Desc	Test Result
1.0	LOGIN	Authentication	Base on user type	Pass
			Base on URLs	Pass
			Multi-factor authentication	Fail
		Authorization	Base on user type	Pass
			Base on URLs	Pass

ソフトウェア動作テスト

テストの設計

●テスト項目の洗い出し

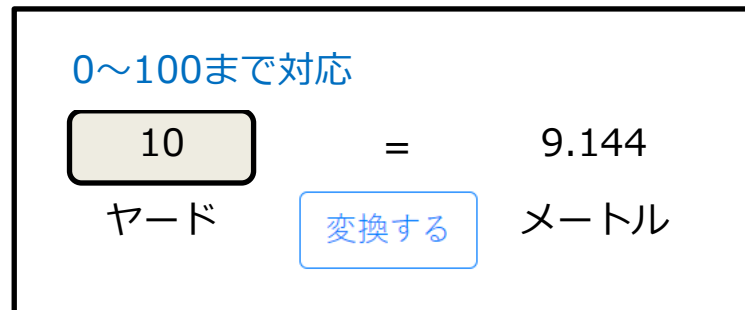
数値を入力として扱う場合、全数をテストすることは小数点以下も入れれば不可能でしょう。条件に引かかる範囲やその境界に着目してテストケースを作成します。

同値分割法

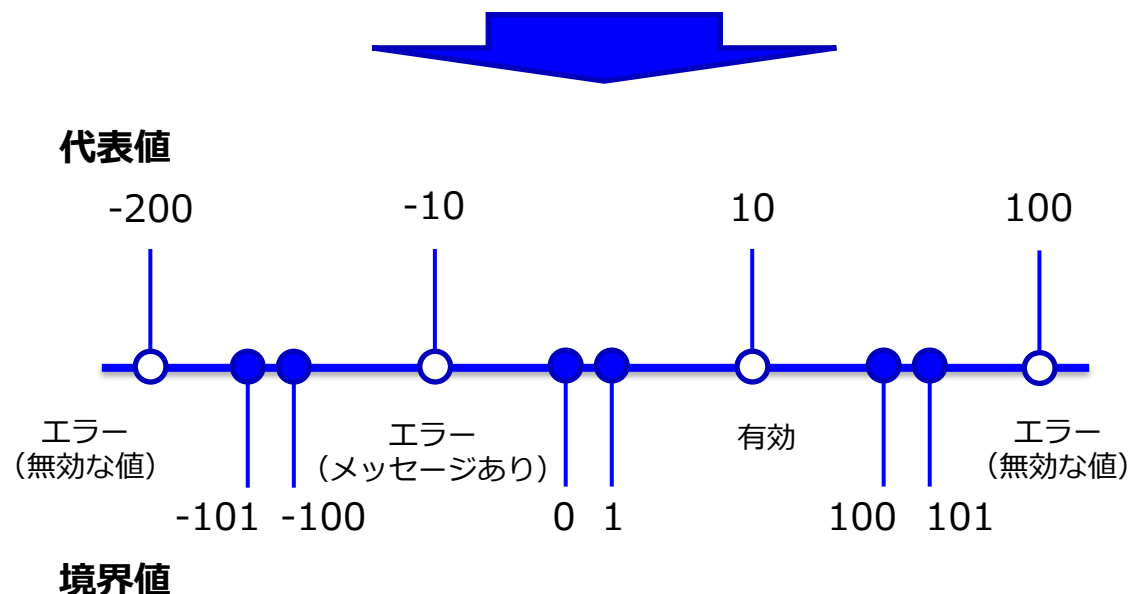
- ・ 閾値内のある値をテストケースの代表値として扱う方法です。
- ・ 同じ条件内であれば、違う値であっても同じリアクションをしますので、テストをするうえでは代表値一つを決めて実施します。

境界値分析

- ・ 入力の上限と下限に対して、リアクションが変わる境界となる限界を超えないが最も許容値に近い値と、限界を超えるが最も許容値に近い値を調べます。



ヤード (y) をメートル (m) に変換するプログラム

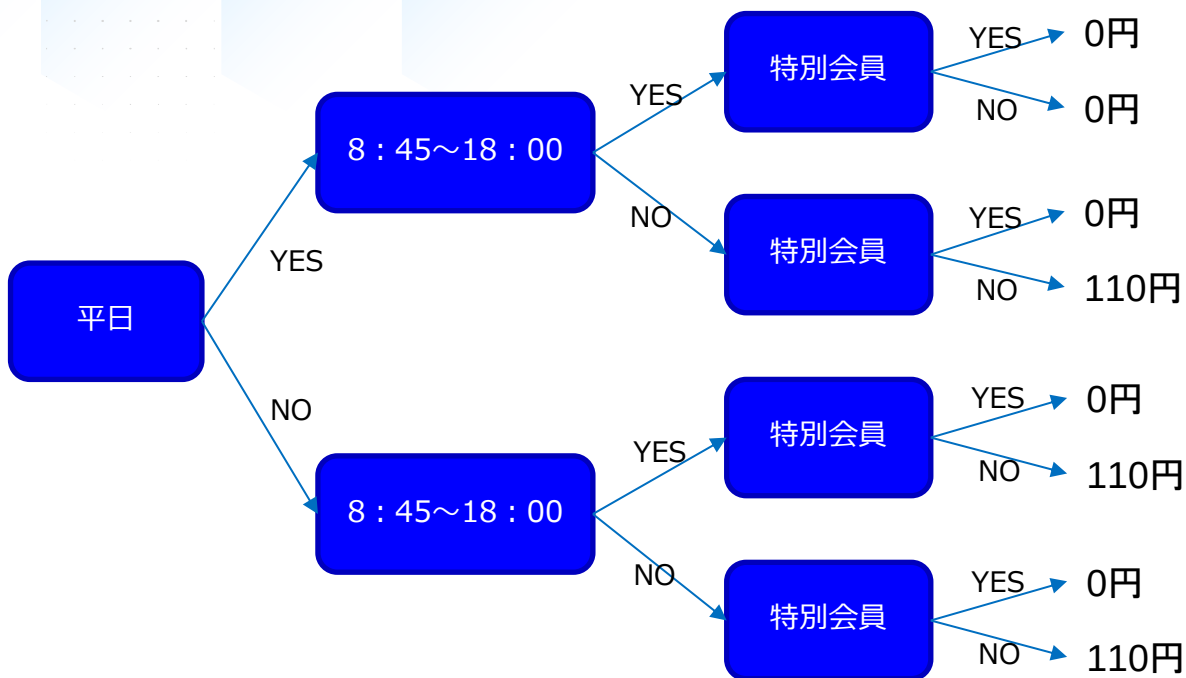


ソフトウェア動作テスト

テストの設計

●テスト項目の洗い出し

- ・ デシジョンテーブルは、入力に対してプログラムがどのように動作するかを表形式にまとめたものです。
- ・ 複雑な条件式の組み合わせからなる論理的構造を表現できます。



	1	2	3	4	5	6	7	8
平日	Y	Y	Y	Y	N	N	N	N
8 : 45 ~ 18 : 00	Y	Y	N	N	Y	Y	N	N
特別会員	Y	N	Y	N	Y	N	Y	N
手数料0円	●	●	●	—	●	—	●	—
手数料110円	—	—	—	●	—	●	—	●

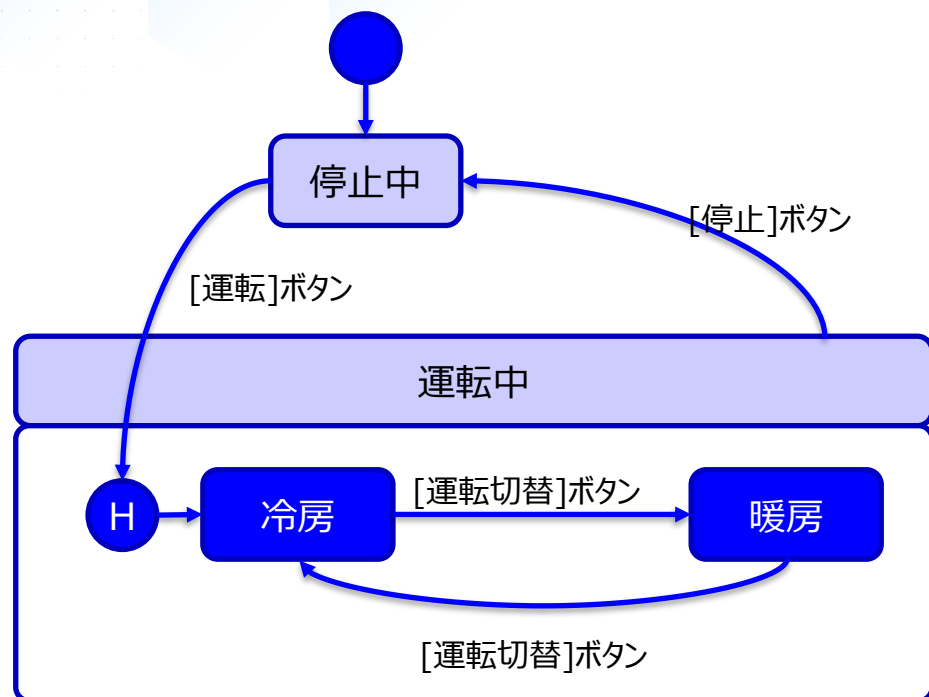
ATM手数料のデシジョンテーブル

ソフトウェア動作テスト

テストの設計

●テスト項目の洗い出し

- 状態遷移図は、状態が遷移する様子を状態を示すノードと矢印で表現したものです。
- Webサイトやゲーム、運転モードが切り替わるデバイスなどのいくつかの状態を持つものがテスト対象である場合には、状態遷移図を明確にしてからテストケースを洗い出します。



遷移前状態	トリガー	遷移先状態
停止中	[運転]ボタン	冷房
停止中	[停止]ボタン	—
停止中	[運転切替]ボタン	—
冷房	[運転]ボタン	—
冷房	[停止]ボタン	停止
冷房	[運転切替]ボタン	暖房
暖房	[運転]ボタン	—
暖房	[停止]ボタン	停止
暖房	[運転切替]ボタン	冷房

ソフトウェア動作テスト

テストの設計

●テストカバレッジ

- テストカバレッジ分析は、ブラックボックステストの設計手法の一つで、アプリケーション全体に対して、どのくらいのコードがテストされたかを特定します。
- テストの不足により、発見されるべきであったバグやセキュリティ問題が存在する可能性のある重大なギャップがないことを確認します。
- アプリケーション処理は、命令処理と条件分岐や繰り返しの連続です。そのため、各分岐によって、テストの対象が増えることになります。

$$\text{網羅率} = \frac{\text{テストされた項目数}}{\text{全テスト項目数}}$$

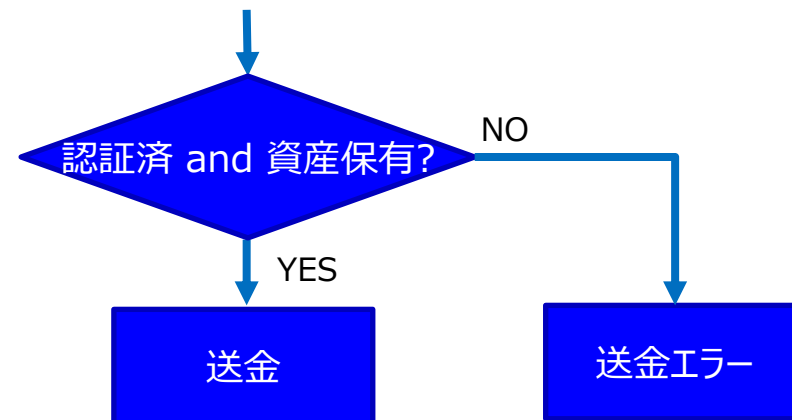
網羅率の計算式

ソフトウェア動作テスト

テストの設計

●テストカバレッジ

- 命令網羅率(C0)とは、すべての実行可能命令(ステートメント)のうち、テストで実行された命令の割合です。送信、送金エラー処理などの網羅率を計測します。
- 分岐網羅率(C1)とは、すべての判定条件のうち、テストで実行された判定条件の割合です。条件分岐のYES、NOの網羅率を計測します。
- 条件網羅率(C2)とは、すべての条件のうち、テストで実行された条件の割合です。認証済がtrueかつ資産保有がtrue、認証済がtrueかつ資産保有がfalse、認証済がfalseかつ資産保有がtrue、認証済がfalseかつ資産保有がfalseの4つの網羅率を計測します。



送金プロセスのサンプル

ソフトウェア動作テスト

テストの設計

●テストの効率化

組み合わせテスト/オールペアテスト

- オールペアテストとは、ブラックボックステストの一つで、入力パラメータの各ペアの可能な離散的な組み合わせをすべて実行するようにテストケースを設計するものです。
- あるアプリケーションの動作を、3種類のOS（Windows、Linux、MacOS）、3種類のPHPバージョン（8.1、8.2、8.3）、3種類のDB（MySQL、PostgreSQL、SQLite）で検証するとします。すべての組み合わせをテストケースとすると、27パターン（ $=3 \times 3 \times 3$ ）になります。
- オールペアテストでは、ある2因子の組み合わせによってバグが発生することを想定し、9パターンのみ実施します。

項番	OS	PHP	DB
1	Windows	8.1	MySQL
2	Windows	8.2	PostgreSQL
3	Windows	8.3	SQLite
4	Linux	8.2	SQLite
5	Linux	8.3	MySQL
6	Linux	8.1	PostgreSQL
7	MacOS	8.3	PostgreSQL
8	MacOS	8.2	SQLite
9	MacOS	8.1	MySQL



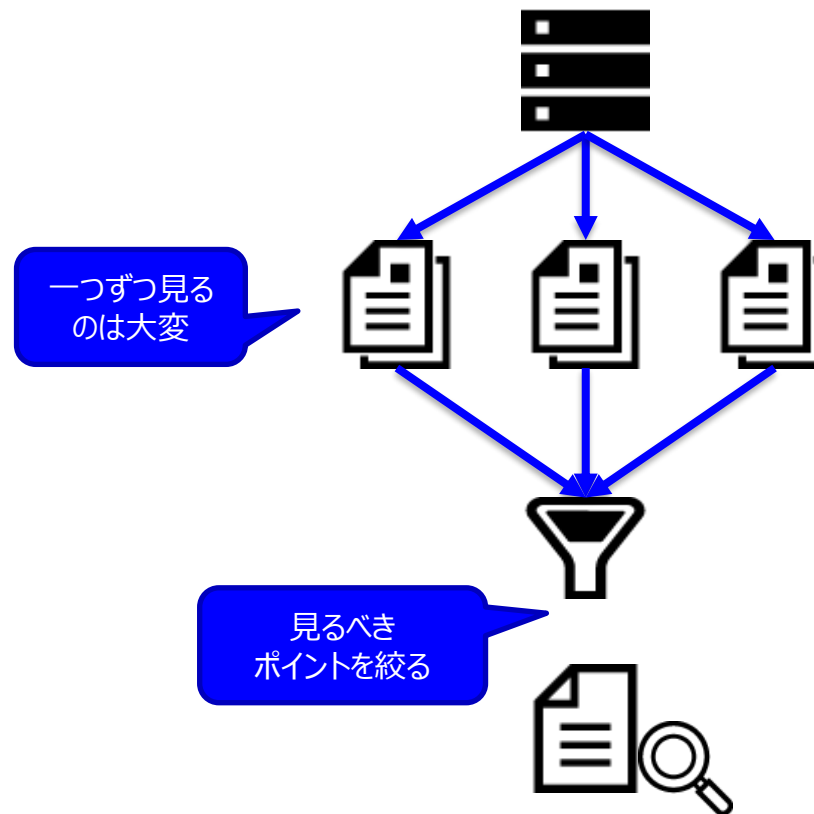
修復と報告

修復と報告

試験結果分析

●テストログレビュー

- テスト時のログはテスト結果を証明するものであるため、大切に保管します。また、テストログは保管する必要がありますが、個人情報や機密情報が含まれるデータを試験に使用せずにテスト用のアカウントなどを利用することで、ログデータにも機密情報が残さないようにします。
- テストが完了したら、テスト時に発生したログを使ってデータを分析する必要があります。テスト時にログは膨大な量になります。
- 定性的なリスク分析の観点においては、可能性や影響が低い場合は一旦放置し、より優先度の高い項目に対応を集中させます。



修復と報告

例外処理

● 例外処理

- 想定しえない状況や入力値によって、ソフトウェアは“例外”が発生します。
- 適正な値以外は受け入れないことは勿論、想定しない結果になった場合でも適切に処理されるように、例外処理としてハンドリングするようにしましょう。
- 例えば、システムで致命的なエラー発生した際に、意図しないプログラムが実行されないように、ブルースクリーンなどの異常状態に遷移させます。

```
try{  
  
    //例外が起こりそうな処理  
  
}catch(Exception e) {  
  
    //例外が発生したときの処理  
  
}
```

例外処理に対するtry catch構文

修復と報告

倫理的開示

● ディスクロージャー

- 一般的には、責任ある開示（Responsible Disclosure）のアプローチを採用します。これは、まずベンダーに通知し、修正パッチがリリースされた後に脆弱性の詳細を公開する方法です。

全面的な開示（Full Disclosure）

- 脆弱性に関するすべての情報（詳細な技術情報、攻撃手法、影響範囲など）を即座に一般公開する。
- セキュリティコミュニティ全体が情報を共有し、解決策の開発を促進する狙いがある。

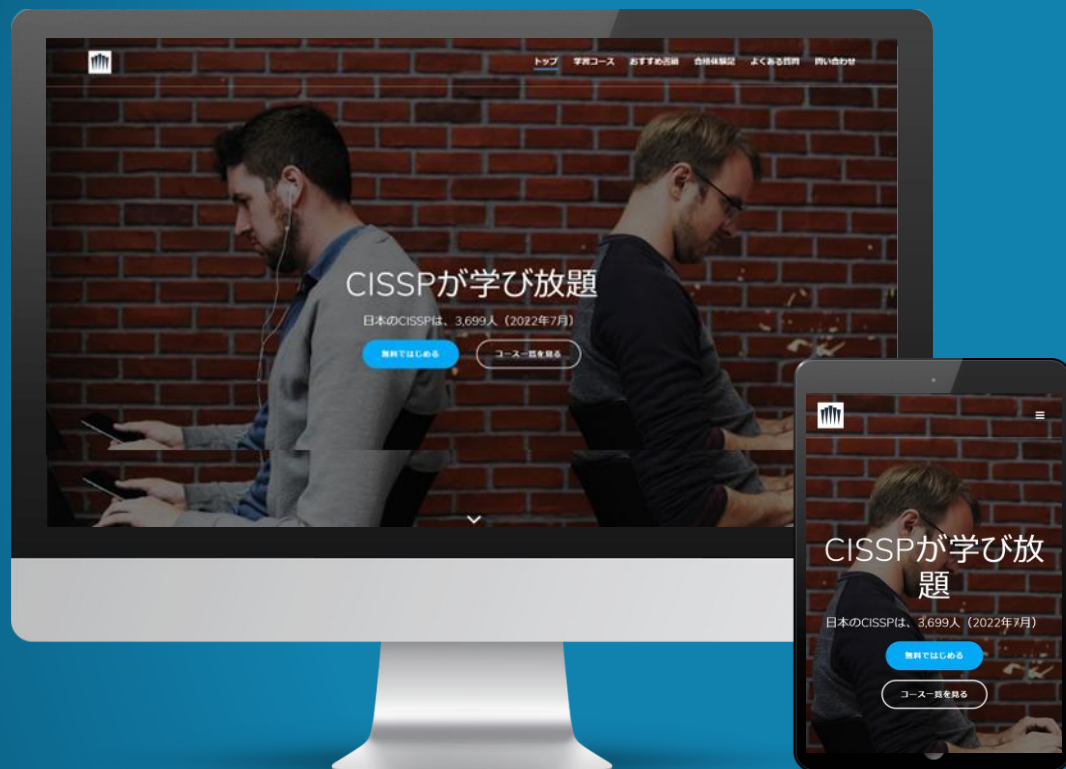
部分的な開示（Partial Disclosure）

- 脆弱性の概要や影響範囲をある程度開示し、詳細情報（具体的な攻撃方法など）は非公開とする。
- 通常、まずベンダーや関係者に非公開で通知し、修正パッチがリリースされた後に情報を一般公開する。

開示しない（Non-Disclosure）

- 脆弱性を発見しても情報を公開しない。または、ベンダーにのみ非公開で通知する。
- ベンダーが修正パッチをリリースした場合でも、脆弱性の詳細が一般に公開されない場合がある。

PIEDPINの紹介



なぜセキュリティを学ぶのか

セキュリティの最も大きな敵は、人はあらゆるものに慣れ、そのリスクを考えなくなる習性にあります。セキュリティの学びを通して得られる貴重なものは、あらゆる事象に対して好奇心を持つことであり、それは目まぐるしい現代を生きるのに大いに役立つでしょう。

- EASY DESIGN
- SUPER EFFICIENT
- EVERYWHERE

PIEDPINを選ぶ理由



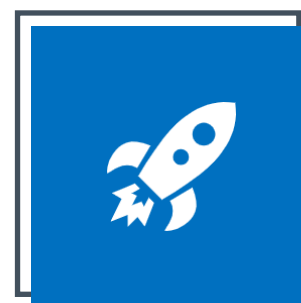
**EASY
DESIGN**

日本語で学べる



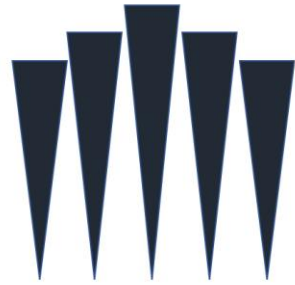
**SUPER
EFFICIENT**

わかりやすい説明



EVERYWHERE

どこからでも学べる環境



THANK YOU

Let's try CISSP!